

Projet du Module 5

Machine Learning

Détection de Transactions Bancaires Frauduleuses

Credit Card Fraud Detection — Dataset 6

Enseignant	Pr. Abdelhamid Fadil
Etudiant	Ahmed Khalil DAOUDI
Programme	Executive Master Data Engineering (MSDE)
Etablissement	Ecole Hassania des Travaux Publics
Année	2025 / 2026

Streamlit App	https://fraud-detection-khalil.streamlit.app/
GitHub	https://github.com/AhmedkhalilDAOUDI/Fraud-detection

1. Compréhension de la Problématique

1.1 Présentation du Dataset

Le dataset utilisé contient des transactions par carte de crédit effectuées en septembre 2013 par des porteurs de cartes européens. Il comporte 284 807 transactions réparties sur deux jours, dont 492 sont des fraudes, représentant seulement 0.17% de l'ensemble des transactions.

Pour des raisons de confidentialité, les features V1 à V28 sont les composantes principales obtenues par ACP (Analyse en Composantes Principales). Seules les variables **Time** (secondes écoulées depuis la première transaction) et **Amount** (montant de la transaction) n'ont pas subi cette transformation.

1.2 Rôle du Machine Learning

La détection manuelle de fraude est impossible à cette échelle. Un modèle ML permet de scorer chaque transaction en temps réel et de lever une alerte automatique quand la probabilité de fraude dépasse un seuil défini. La tâche ML est une **classification binaire** :

- Class 0 → Transaction légitime
- Class 1 → Transaction frauduleuse

1.3 Défi Principal — Déséquilibre des Classes

Constat clé : Déséquilibre sévère — 99.83% légitime vs 0.17% fraude. Un modèle qui prédit systématiquement 'légitime' atteindrait 99.83% d'accuracy sans détecter une seule fraude. L'accuracy est donc une métrique trompeuse. **Métrique principale retenue : PR-AUC (Precision-Recall AUC).**

1.4 Etat de l'Art

Les approches utilisées dans la littérature pour ce problème :

- Régression logistique comme baseline
- Forêts aléatoires et Gradient Boosting pour capturer les non-linéarités
- Isolation Forest et Autoencoders pour l'approche détection d'anomalies
- SMOTE et class_weight pour gérer le déséquilibre des classes
- Optimisation du threshold de décision pour maximiser le F1

2. Analyse Exploratoire des Données (EDA)

2.1 Vue d'ensemble du Dataset

Caractéristique	Valeur
Nombre de lignes	284 807
Nombre de colonnes	31
Valeurs manquantes	0
Features numériques	30 (Time, V1-V28, Amount)
Variable cible	Class (0 ou 1)
Type de problème	Classification binaire

2.2 Distribution de la Variable Cible

Classe	Nombre	Pourcentage
0 — Légitime	284 315	99.83%
1 — Fraude	492	0.17%
Total	284 807	100%

Déséquilibre sévère : Le dataset présente un ratio de 577:1 entre les transactions légitimes et frauduleuses. Ce déséquilibre justifie l'utilisation de PR-AUC comme métrique principale et de SMOTE pour rééquilibrer les classes lors de l'entraînement.

2.3 Analyse des Features

Features V1-V28 : Composantes PCA anonymisées. La transformation PCA garantit l'absence de valeurs manquantes et une normalisation implicite.

Time : Varie entre 0 et 172 792 secondes. Aucun pattern temporel fort n'a été identifié entre fraudes et transactions légitimes.

Amount : Les fraudes ont tendance à avoir des montants plus faibles. Médiane fraude $\approx$ 9.25€ vs médiane légitime $\approx$ 22€.

2.4 Corrélations avec la Target

Feature	Corrélation	Direction
V17	-0.326	Négative (forte)

V14	-0.303	Négative (forte)
V12	-0.261	Négative (forte)
V10	-0.217	Négative
V11	+0.155	Positive
V4	+0.133	Positive
V2	+0.091	Positive

Les features V14 et V17 présentent les corrélations négatives les plus fortes avec la fraude. Ces features seront confirmées comme les plus importantes par le modèle final lors de l'analyse d'importance des features.

3. Pré-processing des Données

3.1 Etapes Non-Nécessaires

Etape	Raison
Imputation	Aucune valeur manquante (dataset propre par construction)
Encodage	Toutes les features sont déjà numériques
Traitement outliers V1-V28	PCA normalise implicitement les données
Feature Engineering	Features PCA déjà optimisées

3.2 Split Train/Test

Split stratifié 80/20 avec **stratify=y** pour préserver le ratio 99.83/0.17 dans les deux parties.

Ensemble	Taille	Fraudes	Pourcentage
Train	227 845	394	0.17%
Test	56 962	98	0.17%

3.3 Standardisation

StandardScaler appliqué **uniquement sur Time et Amount**. V1-V28 sont déjà normalisés par PCA. Le scaler est fitté sur le train uniquement pour éviter tout data leakage.

Formule : $\text{valeur_scalée} = (\text{valeur} - \text{moyenne}) / \text{écart-type} \rightarrow \text{moyenne} = 0, \text{std} = 1$

3.4 SMOTE

Synthetic Minority Oversampling Technique — appliqué uniquement sur les données d'entraînement pour éviter toute contamination du test set.

	Légitime	Fraude	Total
Avant SMOTE	227 451	394	227 845
Après SMOTE	227 451	227 451	454 902

Règle critique : SMOTE est appliqué uniquement sur X_train. Le test set reste intact avec le ratio réel 99.83/0.17 pour une évaluation honnête des performances.

4. Construction des Modèles ML

4.1 Algorithmes Testés

13 algorithmes représentatifs de toutes les familles ML ont été entraînés et évalués. La métrique principale de comparaison est la **PR-AUC**.

Famille	Algorithmes
Linéaire	Logistic Regression, LDA, SGD Classifier
Distances	KNN
Arbres simples	Decision Tree
Bagging (Ensemble)	Random Forest, Extra Trees
Boosting (Ensemble)	AdaBoost, XGBoost, LightGBM, CatBoost
Probabiliste	Naive Bayes
Séparation maximale	SVM (entraîné sur desktop dédié)

4.2 Résultats Comparatifs

Modèle	PR-AUC	ROC-AUC	Recall	Precision	F1	Temps
Extra Trees	0.8753	0.9735	0.8469	0.9121	0.8783	7.95s
Random Forest	0.8747	0.9731	0.8367	0.8454	0.8410	25.33s
CatBoost	0.8574	0.9805	0.8878	0.5437	0.6744	8.55s
XGBoost	0.8462	0.9783	0.8571	0.4641	0.6022	0.87s
LightGBM	0.8110	0.9520	0.8673	0.4913	0.6273	1.70s
AdaBoost	0.7976	0.9734	0.9082	0.0418	0.0799	103s
Logistic Reg.	0.7249	0.9699	0.9184	0.0580	0.1092	1.69s
KNN	0.6333	0.9536	0.8776	0.4649	0.6078	17.79s
Gradient Boost.	0.6885	0.9807	0.8980	0.1055	0.1888	>4h
SVM	0.6087	0.9706	0.8878	0.0844	0.1541	>4h
LDA	0.2664	0.9553	0.8673	0.0967	0.1740	0.80s
Decision Tree	0.2647	0.8865	0.7755	0.3408	0.4735	32.35s
Naive Bayes	0.0860	0.9644	0.8776	0.0550	0.1035	0.28s

* Lignes bleues = Top 3 modèles candidats sélectionnés pour le tuning

4.3 Note sur SVM et Gradient Boosting

SVM et Gradient Boosting ont été entraînés sur une machine dédiée (AMD Ryzen 9 5900X, 64GB RAM, RTX 3080 — 24 threads). Sur la machine principale (MacBook 8GB RAM), ces deux modèles ont nécessité plus de 4 heures d'exécution. Malgré ce temps de calcul important, leurs PR-AUC restent inférieurs aux modèles bagging (0.61 et 0.69 respectivement) — confirmant qu'ils ne font pas partie des candidats au tuning.

5. Tuning des Modèles Candidats et Choix du Modèle Final

5.1 Sélection des 3 Modèles Candidats

Les 3 modèles avec la meilleure PR-AUC ont été sélectionnés :

Rang	Modèle	PR-AUC	Justification
1	Extra Trees	0.8753	Meilleur PR-AUC + Precision 0.91
2	Random Forest	0.8747	Très proche + Precision équilibrée
3	CatBoost	0.8574	3eme PR-AUC + Recall 0.89

5.2 Méthode de Tuning

RandomizedSearchCV avec 10 itérations aléatoires parmi l'espace des hyperparamètres.

Validation croisée StratifiedKFold (3 folds) sur données réelles sans SMOTE pour éviter le data leakage.

Correction importante : Le premier tuning sur données SMOTE a produit PR-AUC CV = 1.0000 (score parfait artificiel). Les points synthétiques SMOTE se retrouvaient dans les folds train et validation simultanément. Correction : tuning sur `X_train_scaled` (données réelles) uniquement.

5.3 Résultats Avant/Après Tuning

Modèle	PR-AUC Avant	PR-AUC Après	Gain	Recall	Precision
Extra Trees	0.8753	0.8808	+0.0055	0.8469	0.9121
Random Forest	0.8747	0.8759	+0.0012	0.8367	0.8283
CatBoost	0.8668	0.8499	-0.0169	0.8571	0.5793

5.4 Optimisation du Threshold

Le threshold par défaut de 0.50 est générique. L'optimisation cherche le threshold qui maximise le F1-score sur la courbe Precision-Recall.

Paramètre	Valeur
Threshold par défaut	0.50
Threshold optimal	0.5767

F1 avec threshold default	0.8783
F1 avec threshold optimal	0.8877
Gain F1	+0.0094

5.5 Modèle Final — Extra Trees (Résultats sur Test Set)

Métrique	Valeur	Interprétation
PR-AUC	0.8808	Très bon — fourchette haute industrie (0.70-0.90)
ROC-AUC	0.9805	Excellent
Recall	0.8469	83 fraudes détectées sur 98
Precision	0.9326	93.26% des alertes sont de vraies fraudes
F1 Score	0.8877	Meilleur équilibre Precision/Recall
Fausses alarmes	6 / 56 864	Quasi imperceptible
Fraudes ratées	15 / 98	Coût acceptable

5.6 Matrice de Confusion

	Prédit Légitime	Prédit Fraude
Réel Légitime	56 858	6
Réel Fraude	15	83

5.7 Importance des Features

Rang	Feature	Importance	Note
1	V14	0.156	La plus discriminante — corrélation forte avec fraude
2	V12	0.099	Confirmée par l'EDA
3	V4	0.094	Corrélation positive forte
4	V17	0.089	Corrélation négative forte (EDA)
5	V11	0.085	Top 5 corrélation positive
15	Time	0.015	Contribution modeste

6. Déploiement du Modèle

6.1 Sérialisation du Pipeline

Le pipeline complet a été sérialisé avec **joblib** en 3 fichiers :

Fichier	Contenu
final_model.joblib	Modèle Extra Trees entraîné (151 MB)
scaler.joblib	StandardScaler fitté sur X_train
best_threshold.joblib	Threshold optimal (0.5767)

6.2 Application Streamlit

Application interactive déployée sur **Streamlit Cloud**.

- Saisie des 30 features (Time, V1-V28, Amount)
- Chargement automatique du modèle depuis Google Drive
- Affichage de la probabilité de fraude en temps réel
- Threshold optimisé appliqué automatiquement
- Bouton 'Charger exemple fraude' pour démonstration

URL : <https://fraud-detection-khalil.streamlit.app/>

6.3 Application Flask (Bonus)

Application web Flask déployée sur **Render** — même fonctionnalités que l'app Streamlit, construite avec HTML/CSS/JavaScript natif.

- API REST — endpoint POST /predict retournant JSON
- Interface web responsive avec design fintech premium
- Même modèle et threshold que l'app Streamlit

6.4 Structure du Dépôt GitHub

<https://github.com/AhmedkhalilDAOUDI/Fraud-detection>

Fichier/Dossier	Description
notebooks/notebook.ipynb	Pipeline ML complet avec commentaires
app.py	Application Streamlit
flask_app/	Application Flask + templates HTML

requirements.txt	Dépendances Streamlit
readme.md	Documentation complète du projet
rapport.pdf	Ce rapport

7. Conclusion

Ce projet a permis de mettre en oeuvre un pipeline ML complet pour la détection de transactions frauduleuses, de la compréhension du problème jusqu'au déploiement en production.

Contributions principales :

- Gestion rigoureuse du déséquilibre sévère (0.17% de fraudes) avec SMOTE et `class_weight='balanced'`
- Sélection de PR-AUC comme métrique principale — plus honnête que ROC-AUC sur données déséquilibrées
- Evaluation de 13 algorithmes couvrant toutes les familles ML classiques
- Optimisation du threshold de décision pour maximiser le F1 (0.50 → 0.5767)
- Déploiement sur deux plateformes : Streamlit Cloud et Render (Flask)

Résultats finaux — Extra Trees :

PR-AUC	Recall	Precision	F1	Fausse alarmes
0.8808	84.69%	93.26%	0.8877	6 / 56 864

Performance en contexte industrie : Les systèmes de détection de fraude en production tournent généralement entre 0.70 et 0.90 de PR-AUC. Notre modèle Extra Trees avec PR-AUC = 0.8808 se situe dans la fourchette haute, avec une Precision de 93.26% — seulement 6 fausses alarmes sur 56 864 transactions légitimes.